

Article

Extraction of Minimal Set of Traffic Features Using Ensemble of Classifiers and Rank Aggregation for Network Intrusion Detection Systems

Jacek Krupski , Marcin Iwanowski  and Waldemar Graniszewski 

Institute of Control and Industrial Electronics, Warsaw University of Technology, ul. Koszykowa 75, 00-662 Warszawa, Poland; marcin.iwanowski@pw.edu.pl (M.I.); waldemar.graniszewski@pw.edu.pl (W.G.)

* Correspondence: jacek.krupski.dokt@pw.edu.pl

Abstract: Network traffic classification models, an essential part of intrusion detection systems, need to be as simple as possible due to the high speed of network transmission. One of the fastest approaches is based on decision trees, where the classification process requires a series of tests, resulting in a class assignment. In the network traffic classification process, these tests are performed on extracted traffic features. The classification computational efficiency grows when the number of features and their tests in the decision tree decreases. This paper investigates the relationship between the number of features used to construct the decision-tree-based intrusion detection model and the classification quality. This work deals with a reference dataset that includes IoT/IloT network traffic. A feature selection process based on the aggregated rank of features computed as the weighted average of rankings obtained using multiple (in this case, six) classifier-based feature selectors is proposed. It results in a ranking of 32 features sorted by importance and usefulness in the classification process. In the outcome of this part of the study, it turns out that acceptable classification results for the smallest number of best features are achieved for the eight most important features at 95.3% accuracy. In the second part of these experiments, the dependence of the classification speed and accuracy on the number of most important features taken from this ranking is analyzed. In this investigation, optimal times are also obtained for eight or fewer number of the most important features, e.g., the trained decision tree needs 0.95 s to classify nearly 7.6 million samples containing eight network traffic features. The conducted experiments prove that a subset of just a few carefully selected features is sufficient to obtain reasonably high classification accuracy and computational efficiency.

Keywords: cybersecurity; Internet of Things; machine learning for cybersecurity; intrusion detection systems; decision trees; network traffic features; ensemble feature selection



Citation: Krupski, J.; Iwanowski, M.; Graniszewski, W. Extraction of Minimal Set of Traffic Features Using Ensemble of Classifiers and Rank Aggregation for Network Intrusion Detection Systems. *Appl. Sci.* **2024**, *14*, 6995. <https://doi.org/10.3390/app14166995>

Academic Editors: Xavier A. Larriva-Novo and Andres Marin Lopez

Received: 15 July 2024
Revised: 2 August 2024
Accepted: 5 August 2024
Published: 9 August 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Network traffic is a target of cyberattacks of various kinds, which generate gigantic costs, e.g., according to some estimates, cyberattacks cost from PLN1 trillion to as much as PLN 5 trillion a year globally [1]. Consequently, their detection is one of the crucial challenges of communication systems, resulting in the development of intrusion detection systems (IDSs). These systems are an important part of network security and are responsible for blocking unwanted traffic, like DOS [2] or attacks on web applications [3]. They are one of the main tools of network administrators to ensure network security [4]. IDSs usually use features computed online based on the registered data stream. These usually refer to some value of fields extracted from the datastream and/or times/durations or quantities computed on the fly. Such data make up the input of the machine learning model that classifies the data sample as benign or harmful traffic—usually, several classes of the latter are distinguished. Machine learning models are trained on pre-registered datasets containing the traffic samples. However, finally, one prepares them to apply in the classification of real network traffic. The transmission speed of modern networks

forces an appropriate processing speed—the classifiers in this domain must be fast and able to finally run in real-time. This—in turn—requires the optimization of all the aspects of the traffic classifier: the number of features used, the classification algorithm, and its implementation. In this paper, the problem of choosing the smallest possible set of traffic features and preserving high classification accuracy is discussed. Such features are then ready for the classifier to use, e.g., the fast decision-tree model. This investigation proposes a rank aggregation approach based on the ensemble approach employing multiple methods: random forests, extra trees, adaboost, passive aggressive algorithm, support vector machine, and ridge classifier. They are used as feature selectors, producing a list of features sorted by relevance (ranked list). To combine the ranks produced by various methods, a linear combination of ranks with accuracies of appropriate classifiers are used as weights—the latter results in the aggregated ranking of features. By simple rank thresholding, one can then obtain the set of features that can be applied to train the classifier, the quality of which depends on the number of features chosen. Three classic measures are used to measure the quality: accuracy, precision, and recall.

Feature selection has been widely used for a variety of research problems, from blood composition analysis to checking text in emails, up to processing images containing fingerprints [5]. Another application of feature selection is its use for processing network traffic for IDS purposes [6] as this work performs. In this investigation, the combination of three decision-tree-based and three linear classifiers is used to perform feature aggregation in their selection process. The goal is to finally provide a set of features that may be used to induct a target decision tree, the backbone of a fast classifier able to work in real-time in intrusion detection systems. As the target machine learning model, decision tree classifiers are chosen. They are constructed using previously selected features. They are characterized by several properties that make them an optimal classifier for IDSs [7–9]. First, compared with other classification models, they are computationally less intensive, allowing for faster network traffic classification. It is essential in real-time applications where decisions must follow the data flow. They are also easy to interpret [10], which is crucial in network traffic analysis, where understanding the reasoning behind a classification is as important as the classification itself. In addition, they are robust to noise in the data, can handle non-linear relationships, and are easy to update as new data are collected.

Because the machine learning model requires the data to perform training efficiently [11], datasets were carefully chosen to be used in these experiments. The proper choice of data is crucial to obtain a useful classification model in all machine learning application fields [12–14], including cybersecurity. As the network traffic data vary and might differ in various types of networks, there are datasets—UNSW-NB15, BoT-IoT, ToN-IoT, and CIC-CSE-IDS2018—that collect various types of internet traffic, which have been combined into a single NF-UQ-NIDS-v2 dataset [15]. The latter dataset comprises almost 76 million records and 43 features, making it a good choice for training machine learning-based intrusion detection models.

The key contributions of this study are the following:

- The novel scheme of traffic features ordering, consisting of the selection of classifiers, their hyperparameter optimization, individual feature rankings of the best variant of classifiers, and finally, weighted feature ranking, results in an ultimate ordered set of features.
- The analysis of the NF-UQ-NIDS-v2 datasets' features was sorted using the proposed scheme in view of their influence on the decision-tree-based classification, considering both accuracy and computational efficiency.
- The final decision-tree-based classifier with a mechanism allowing for optimizing the speed-to-accuracy rate. By selecting various thresholds on the ordered feature list, one may increase the accuracy, lowering the inference speed and reversely.

The rest of the paper is organized as follows. Section 2 is the literature survey. Section 3 introduces the dataset used with their proper features. In Section 4, there is a description of the proposed feature selection approach. Section 5 presents and discusses the obtained results, whereas Section 6 concludes the paper.

2. Previous Works

The methods for producing the optimal number of relevant features have been the subject of much machine learning (ML) research. This is due to the crucial role of features in supervised learning. As this research focuses on network intrusion detection systems, the first subsection of this chapter is about the aspect of feature engineering.

The second subsection is devoted to the datasets that are used to train the ML models.

2.1. Selecting Network Traffic Features

Network traffic can be represented as a set of features, which may reflect important aspects of traffic events, like attacks [15].

In the middle of 1990, the NetFlow protocol was developed by Cisco Systems for accounting purposes. With the timeline, it also became implemented for other tasks, such as billing, application and protocol monitoring, and many others [16]. NetFlow monitors traffic flows through network devices such as a switch or router and generates traffic features suitable for the analysis. Other vendors like Juniper Networks developed their proprietary protocol—similar to NetFlow—jFlow. There are several versions of the NetFlow, jFlow, and other flow-based protocols; therefore, the Internet Engineering Task Force (IETF) developed the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information [17].

Managed network devices like switches and routers can have implemented flow-based exporters that record traffic features and export them to flow collectors. In the case when network traffic has already been recorded as raw traffic and saved in a PCAP file format, those traffic features can be extracted offline with dedicated software.

The issues that come with the data are mostly the redundancy and importance of features. Therefore, a wide variety of feature selection methods are proposed. Works devoted to this topic distinguish the following techniques: wrapper, embedded, filter, and hybrid [18,19].

Wrapper methods are all about submitting different combinations of feature subsets to check their classification results by machine learning algorithms [20]. Then, the subset with the best results defines the ideal batch of features. Wrapper methods are said to be inadequate for massive datasets due to the high demand for computational power.

Embedded methods can utilize two ways to determine the best features. Firstly, some methods can adopt machine learning algorithms, e.g., random forest, to classify samples according to each feature. Later, the computed results are compared to find the best set of features. The second group of embedded methods is based on regularization. It means that during computing, penalties are added to features in order to prevent overfitting [21]. In the end, features with a smaller cumulative value of penalties are chosen.

Looking for the best features with the advantage of filter methods consists of calculating the coefficient value for each feature to create a ranking [22]. Then, one can decide on the condition on which features with worse coefficient values would be cut off.

The last technique for feature selection is hybrid methods. They are any combination of the previously mentioned methods.

Selecting the right features for an intrusion detection system (IDS) is crucial for its performance and accuracy. The selection process often involves understanding the nature of network traffic and the types of attacks the IDS is intended to detect. There are several types of features. Basic features [23] include IP addresses, port numbers, protocol types, and packet sizes. They are easy to capture and provide a foundational understanding of the traffic. Content features [24] involve examining the payload of packets for malicious content or anomalies. They can include signatures of known attacks, unusual byte sequences, or suspicious keyword usage. Time-based features [25] focus on the timing of the traffic, like the duration of the connection, the time between packets, or the frequency of connections to a particular IP address. Finally, behavioral features [26] analyze the behavior of hosts or networks over time, such as changes in traffic patterns, unusual outbound connections, or sudden spikes in data transfer.

Scientists utilize widely mentioned feature selection methods in their research. For instance, in [10], the authors performed selection using a hybrid method for the classification of traffic features. The work was based on the top 15 features from the UNSW-NB15 dataset.

Two filter methods, ANOVA and chi-square test, were used in parallel to select the best features [27]. The first was analyzing the variances, whereas the second picked dependent features. Researchers used these methods on their own traffic. In their tests, the most important features were the number of bytes sent and received.

Another research work used two feature selection methods, one after the other: an embedded method, naive Bayes, and a filter method, *t*-test [28]. The paper started with nine features and concluded that two features should be rejected.

Researchers working on network traffic classification widely utilize different selection methods to create a combined ranking of features. Such a method is called an ensemble feature selection [29,30]. A brief comparison of the ensemble feature selection concept proposed in this publication, along with methods from the literature review, is included in Table 1.

Table 1. Comparison of ensemble feature selection concepts for cybersecurity purposes.

Method	Wrapper	Embedded	Filter	Hybrid
SVM Rank [29]		1	4	1
Voting [30]	2	1	2	
Majority voting [31]			5	
Average importance factor [32]			2	
Naive Bayes Rank [33]			2	
Weighted average [34]		1	2	
Our approach		6		

In order to classify network traffic, the results of six feature selection methods: an embedded method (perceptron), four filter methods (chi-square, minimum redundancy, maximum relevance, and ReliefF) and a hybrid method (recursive feature elimination for SVM) were combined using SVM with a radial-basis function [29]. A combination of five feature selection methods—two wrapper methods (stability selection and recursive feature elimination), an embedded method (RF), and two filter methods (mean decrease impurity, and chi-square)—using the voting of selected classifiers was conducted [30]. The results of the five filter methods (information gain, gain ratio, chi square, symmetric uncertainty, and relief) were combined using summing [31]. The authors of [32] proposed combining two filter methods: information gain and symmetrical uncertainty. Due to the tests, the method achieved promising results. The idea of combining two filter methods was proposed while classifying Android traffic [33]. In the examination, both information gain and chi-square test showed that the initial set of 22 features should be minimized to only nine features. To enhance IDS systems, a feature ranking consisting of three selection methods was conducted [34]. For this purpose, two filter methods, the chi-square test and information gain, were used along with the extra trees embedded algorithm.

2.2. Network Traffic Datasets

Detailed revision of the available datasets for network intrusion detection with special attention to the technical aspects of the data was presented in [35]. The research contained the following criteria for dataset properties: evaluation, recording environment, nature of the data, data volume, and general information. In the first group, there was meaningful information, like labeling or the dataset being balanced, which was contained in the evaluation. The recording environment group informed us if the traffic was emulated, e.g., inside a university network, or captured in a real-world scenario, e.g., honeypot. The nature of the data was about the data format, the existence of metadata, and anonymity, if any. The paper mainly focused on packet or flow-based traffic. However, some feature-based datasets, like KDD Cup 1999, were also described. The paper concluded that no perfect

network-based dataset exists. However, researchers recommend working with up-to-date, correctly labeled and, ideally, publicly available datasets.

Datasets tested with the advantage of convolutional neural networks were gathered and ranged according to their popularity [36]. From this research article, datasets in the format of features, e.g., UNSW-NB15 and ISCX-IDS-2017, were, among others, highlighted. Special attention was paid to the datasets' size or year of creation and exact studies in which they were tested.

The survey about protecting critical infrastructure mentions twenty datasets used for cybersecurity purposes [37]. The feature-based NF-UQ-NIDS-v2 is up to date when compared with the majority of other datasets.

The usability ranking of datasets is an interesting concept while selecting datasets [38]. The survey continued the idea of [35] and highlighted that some datasets need to be correctly labeled. The authors also focused on the presence of complex attacks and feature sufficiency. In this scientific research, seven popular network traffic datasets were compared, pointing to NF-UQ-NIDS-v2 [15] as the best choice for researchers.

Similarly, in survey [39] that compared 16 network traffic datasets, the differentiation of attacks was highlighted. The article described the availability and accuracy of labeling as important dataset properties. Here, NF-UQ-NIDS-v2 was also a recommendation for researchers.

In another review of datasets [40], the authors mainly compared those datasets where there is IoT traffic. Among all, they pointed to NF-UQ-NIDS-v2, which, through a unified set of features, provides the opportunity to compare the classification results of each of its component datasets.

According to the recommendations, such as a dataset being up to date or containing real-world traffic, the choice of the surveys mentioned earlier relating to the top dataset is found to be constructive. Therefore, in this paper, the NF-UQ-NIDS-v2 dataset is utilized. In addition, it is worth noting that this dataset contains an increased number of network attacks compared to a single dataset [21]. Also of great importance is a unified set of features based on the popular Netflow protocol. The aforementioned benefits make NF-UQ-NIDS-v2 a universal dataset for IDS learning purposes.

3. Traffic Features

In this section, NF-UQ-NIDS-v2 is described, and particular attention is paid to the features of network traffic that have been included in this dataset.

3.1. NF-UQ-NIDS-v2 Dataset

As recommended by surveys [38,39], this research uses the dataset NF-UQ-NIDS-v2, which is composed of four publicly available and widely utilized datasets: UNSW-NB15, BoT-IoT, ToN-IoT, and CIC-CSE-IDS2018 [15]. The first three datasets were prepared by the Cyber Range Lab of the University of New South Wales (UNSW) Canberra, and the fourth one by a common project of the Communications Security Establishment and Canadian Institute for Cybersecurity.

UNSW-NB15 has benign traffic as well as nine families of attacks: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms [41]. The traffic was captured using the tcpdump tool, whereas the attacks themselves were generated by the IXIA tool. The network in which the dataset was prepared contains virtual servers, client machines, and a firewall. This dataset size is 0.55 GB [36].

BoT-IoT focuses mainly on the cyber threat of botnets. The dataset's PCAP files with the traffic occupy 69.3 GB [15]. Apart from benign traffic, four attacks can be found: DoS, DDoS, Reconnaissance, and Theft. While creating this dataset, the core notion was to highlight the importance of security in IoT networks [42]. The dataset consists of traffic captured by the Node-Red tool between selected IoT sensors. Authors worked, among others, with a smart fridge or a smart thermostat.

ToN-IoT is about the computer traffic of IoT networks with operating systems' logs and telemetry data [15]. The creators of this dataset were also guided by the idea of improving the security within IoT networks. They captured benign traffic and nine attacks: Scanning, DoS, DDoS, Ransomware, Backdoor, Injection Attack, XSS, Password Cracking Attack, and Man In The Middle [43]. These attacks were conducted on IoT sensors, like the IoT Global Positioning System tracker, or a remotely activated garage door.

The last dataset, CIC-CSE-IDS2018, contains benign traffic and six types of attacks: Brute Force, Heartbleed, Botnet, DoS, DDoS, Exploiting vulnerable software (e.g., backdoors), and web attacks such as SQL injection, Cross-Site Scripting, or Brute Force over HTTP [44]. The architecture of the used network was immensely expanded, as it contained three servers, ten computers (Windows, Linux and Mac) and a firewall.

Two datasets, CIC-CSE-IDS2018 and UNSW-NB15, are characterized by a high percentage of benign traffic samples included. The percentage of attack samples is relatively small (less than 20% to 80% of normal traffic). Radically different are the two remaining datasets, where normal traffic represents just a few percentages of all samples [15].

The original versions of the four datasets contain only a few common features, e.g., IPv4 addresses. Noticeably, more features, like the outgoing number of bytes, are less popular and occur only in some datasets. A list of features present in particular original datasets is shown in Table 2. When unifying datasets, the authors of [15] decided to regenerate network traffic features from original raw traffic (PCAP files) with the advantage of the nProbe tool. As a result, a new set with a total number of 75,987,976 records was generated. The NF-UQ-NIDS-v2 dataset not only merges the mentioned four datasets but also provides an extended and unified set of network traffic features when compared to their standard versions.

Table 2. Example features present in particular original datasets.

Feature	NB15	BoT	ToN	IDS2018	NIDS-v2
IPv4 source address	✓	✓	✓	✓	✓
IPv4 source port number	✓	✓	✓	✓	✓
IPv4 destination address	✓	✓	✓	✓	✓
IPv4 destination port number	✓	✓	✓	✓	✓
Incoming number of packets	✓	✓	✓	✓	✓
Outgoing number of packets	✓	✓	✓	✓	✓
Incoming number of bytes	✓	✓	✓		✓
Outgoing number of bytes	✓	✓	✓		✓
Flow duration	✓	✓	✓		✓
Protocol	✓	✓	✓		✓
Transaction state	✓	✓	✓		
Src to dst bytes/s		✓		✓	✓
Dst to src bytes/s		✓		✓	✓
Mean size of outgoing packet	✓			✓	
Mean size of incoming packet	✓			✓	
Start time of the traffic	✓	✓			
Pipelined depth into HTTP connection	✓		✓		
DNS query type			✓		✓
Length of the smallest flow				✓	✓
Length of the largest flow				✓	✓

Conducting experiments on the whole NF-UQ-NIDS-v2 demands relatively high computational power. Therefore, some research teams decided to work only on portions of the dataset. In [45], intrusion detection with seven machine learning algorithms was tested on two subsets of NF-UQ-NIDS-v2: 100,000 and 10 mln records. In different research devoted to securing the telecommunication industry [46], deep learning methods were tested on 100,000 dataset records. Two subsets of NF-UQ-NIDS-v2, i.e., NF-BoT-IoT-v2 and NF-CSE-CIC-IDS2018-v2, constituted the data on which the detection of DDoS attacks on IoT was checked [47]. Another study explored the idea of using representational learning

for IDS purposes [48]. The research used 180,000 samples from two datasets: NF-UQ-NIDS-v2 and ToN-IoT. Unlike most papers that check their solutions on a set of NF-UQ-NIDS-v2, this study works on the entire set.

3.2. Traffic Features in NF-UQ-NIDS-v2

In the NF-UQ-NIDS-v2 dataset, 43 unified features are proposed. All are listed in Table 3. All of them are related to the popular NetFlow specification. They can be assigned to the following two types:

- Categorical (‘c’ in the second column of Table 3), which contain 14 features. This type includes addresses and specific communication protocols.
- Numerical with the remaining 29 features (‘n’ in the second column of Table 3). This type contains all features that represent numeric data of the traffic, such as average throughput, number of transmitted packets, or the duration of the traffic.

Table 3. Features in NF-UQ-NIDS-v2. Description of each feature follows [15].

Feature	Type	Description	Group
1. IPV4_SRC_ADDR	c	IPv4 source address	II
2. L4_SRC_PORT	c	IPv4 source port number	II
3. IPV4_DST_ADDR	c	IPv4 destination address	II
4. L4_DST_PORT	c	IPv4 destination port number	II
5. PROTOCOL	c	IP protocol identifier byte	I
6. L7_PROTO	c	Application protocol as a number	I
7. IN_BYTES	n	Incoming number of bytes	IV
8. IN_PKTS	n	Incoming number of packets	IV
9. OUT_BYTES	n	Outgoing number of bytes	IV
10. OUT_PKTS	n	Outgoing number of packets	IV
11. TCP_FLAGS	c	Cumulative of all TCP flags	III
12. CLIENT_TCP_FLAGS	c	Cumulative of all client TCP flags	III
13. SERVER_TCP_FLAGS	c	Cumulative of all server TCP flags	III
14. FLOW_DURATION_MILLISECONDS	n	Flow duration in milliseconds	V
15. DURATION_IN	n	Incoming stream duration in milliseconds	V
16. DURATION_OUT	n	Outgoing stream duration in milliseconds	V
17. MIN_TTL	n	Minimal flow TTL	V
18. MAX_TTL	n	Maximal flow TTL	V
19. LONGEST_FLOW_PKT	n	Longest packet (bytes) of the flow	IV
20. SHORTEST_FLOW_PKT	n	Shortest packet (bytes) of the flow	IV
21. MIN_IP_PKT_LEN	n	Len of the smallest flow IP packet observed	IV
22. MAX_IP_PKT_LEN	n	Len of the largest flow IP packet observed	IV
23. SRC_TO_DST_SECOND_BYTES	n	Src to dst bytes/sec	V
24. DST_TO_SRC_SECOND_BYTES	n	Dst to src bytes/sec	V
25. RETRANSMITTED_IN_BYTES	n	Number of retransmitted TCP flow bytes (src- dst)	VI
26. RETRANSMITTED_IN_PKTS	n	Number of retransmitted TCP flow packets (src->dst)	VI
27. RETRANSMITTED_OUT_BYTES	n	Number of retransmitted TCP flow bytes (dst->src)	VI
28. RETRANSMITTED_OUT_PKTS	n	Number of retransmitted TCP flow packets (dst->src)	VI
29. SRC_TO_DST_AVG_THROUGHPUT	n	Src to dst average throughput (bps)	V
30. DST_TO_SRC_AVG_THROUGHPUT	n	Dst to src average throughput (bps)	V
31. NUM_PKTS_UP_TO_128_BYTES	n	Packets whose IP size ≤ 128	VII
32. NUM_PKTS_128_TO_256_BYTES	n	Packets whose IP size > 128 and ≤ 256	VII
33. NUM_PKTS_256_TO_512_BYTES	n	Packets whose IP size > 256 and ≤ 512	VII
34. NUM_PKTS_512_TO_1024_BYTES	n	Packets whose IP size > 512 and ≤ 1024	VII
35. NUM_PKTS_1024_TO_1514_BYTES	n	Packets whose IP size > 1024 and ≤ 1514	VII
36. TCP_WIN_MAX_IN	n	Max TCP Window (src->dst)	III
37. TCP_WIN_MAX_OUT	n	Max TCP Window (dst->src)	III
38. ICMP_TYPE	c	ICMP Type $\cdot 256 + \text{ICMP code}$	I
39. ICMP_IPV4_TYPE	c	ICMP Type.	I
40. DNS_QUERY_ID	c	DNS query transaction Id.	I
41. DNS_QUERY_TYPE	c	DNS query type (e.g., 1 = A, 2 = NS etc.)	I
42. DNS_TTL_ANSWER	n	TTL of the first A record (if any)	I
43. FTP_COMMAND_RET_CODE	c	FTP client command return code	I

In general, when designing the traffic classifier, one needs to use features that are related to the properties of the traffic itself rather than influenced or even strictly dependent on the infrastructure used to capture the dataset used for training the classification model. Therefore, to prepare classification models, one should use features that are as infrastructure

independent as possible. Such universally crafted traffic would be perfect for training IDS, which may work in various networks with radically different setups. Unfortunately, this demand for traffic features may be hard to fulfill. Careful analysis of all features led us to divide them into seven groups, collecting features with similar roles and emphasizing the network infrastructure dependence. An assignment of a particular feature to its proper groups is shown in the fourth column of Table 3. The proposed groups of features are as follows:

1. General information about protocols—group I. This group contains details of protocols utilized in the communication, like the type of L7 protocol, type of ICMP, or FTP command return code. In general, not all features in this group are strictly dependent on the network infrastructure, but there are some examples where these connections may occur. For instance, ICMP controls the links within the network, and one of its answers explicitly points out that a destination is unreachable or that the time to obtain the answer has passed. It is similar with DNS queries, which differ depending on the IP protocol version. If IPv4 is in use, DNS will work on A records, whereas for IPv6, AAAA will be in use.
2. Addressing data—group II. While preparing network traffic datasets, authors should anonymize or remove exact IP addresses, as they may bias the machine learning tools. Port numbers may or may not be helpful for research purposes. Some network attacks are strictly connected to exact port numbers in the victim's machines, but the attackers may easily change them. When attacks are emulated in the laboratory network, their addresses are entirely unimportant. However, addressing data could be helpful while working with the attacks captured in the real world. To sum up, it is believed that this sort of data should be omitted in examinations like this one.
3. TCP parameters—group III. In TCP, many parameters are used to establish a session between computers. NF-UQ-NIDS-v2 contains only two of them: cumulative values of TCP flags and TCP window sizes. It appears that these features are moderately connected to the infrastructure where the traffic is emulated. In the DoS or DDoS attacks on TCP session establishing, the so-called SYN flood attack type, an attacker initiates a connection by setting a TCP SYN flag but does not continue the connection acknowledging the server's response, leading to the lack of resources for the legitimate clients. Such attacks can be discovered by analyzing the state of particular initialized sessions.
4. Sent data—group IV. The analyzed dataset has a wide range of features devoted to the volume of sent data, e.g., the incoming number of bytes. Then, there are also lengths of the shortest or longest flows. This sort of feature should be more related to the settings in tools that are used by the attackers rather than the infrastructure itself.
5. Transmission parameters (time, speed, throughput and TTL)—group V. Features in this group represent the traffic characteristics that should not depend on the infrastructure build. Traffic speed or throughput are firmly related to architecture of particular computer networks. A noticeable relation between TTL features and the appropriate traffic class was observed [15]. Therefore, the authors of NF-UQ-NIDS-v2 decided to refrain from taking this feature into research. Most likely, they linked TTL values with differing infrastructures in which the traffic was captured. TTL represents the number of nodes that could be entered by the packet while traveling within the network. Passing each node decreases the TTL value, thus preventing the creation of any loops in the network. Extremely high TTL values may be associated with DoS or DDoS attacks that flood the network with a massive number of packets. Continuing this line of thinking, the times in which flows are delivered through the network also could depend on the construction of a network. Nevertheless, features from this group seem more likely to rely on the network standards instead of the infrastructure.
6. Retransmission parameters—group VI. In this group, retransmission features are placed. These features indicate how many packets did not reach their destination

and had to be resent. Retransmission issues of packets occur randomly with no dependence on the network build.

7. Packet sizes—group VII. This batch contains five features that accumulate chosen packet sizes. As long as network devices have no packet size restrictions, it is assumed that this group is less likely to rely on infrastructure.

4. Methodology

The aim of this research is to check if the number of features could be minimized while preserving good classification quality. Therefore, this investigation utilized feature selection methods, which removed redundant features (attributes) to enhance classifiers' accuracies and improve the inference process in terms of time [20]. A brief summary of the performed research is written in Algorithm 1.

Algorithm 1 A summary of the research steps performed in Section 4

Step 1 (Section 4.1)

```

Save all 43 features in features
from features remove IPV4_SRC_ADDR AND IPV4_DST_ADDR
from features remove L4_SRC_PORT AND L4_DST_PORT
from features remove DNS_QUERY_ID AND ICMP_IPV4_TYPE

calculate Pearson correlation for all features
find all high correlation pairs, where  $\text{abs}(\text{correlation}) \geq 0.9$ 
for each pair in high correlation pairs do
  for each feature in pair do
    calculate sum of all correlations for feature
  end for each
  from each pair choose 1 feature with higher sum of all correlations
  remove chosen feature from features
end for each

```

Step 2 (Section 4.2)

```

conduct grid_search for 6 classifiers (RF, ET, AB, PA, SVM, R)
for each classifier in classifiers do
  for each parameter_1 in range_1 do
    for each parameter_2 in range_2 do
      train classifier(..., parameter_1, parameter_2, ...)
      test classifier(..., parameter_1, parameter_2, ...) AND save test metrics
    end for each
  end for each
  find classifier with highest test metric AND save it in chosen_classifiers
end for each

```

Step 3 (Section 4.3)

```

for each classifier in chosen_classifiers do
  create feature_ranking AND save it
end for each
calculate final rank for each feature acc. to Equation (2)

```

4.1. Preliminary Selection of Features

The first step of this examination performs the feature preselection by rejecting features that are either strictly infrastructure dependent or redundant. From the set of 14 categorical features, four address features (group II) are withdrawn, as well as DNS_QUERY_ID (from group I). The addresses are firmly connected with the infrastructure in which the network traffic was captured—in this case, the capturing setup of the original dataset. They do not represent any general traffic characteristics, and using them may result in biased classification outcomes. The same reason stands behind the removal of DNS query ID. Then, also ICMP_IPV4_TYPE from group I is withdrawn, as it is already captured in ICMP_TYPE.

The next step is to compute the correlation on the remaining traffic features to find and reject the redundant ones. In this examination, valuable information on highly correlated features is obtained. An absolute value of the correlation coefficient greater than 0.9 is achieved by five pairs of features. These are as follows:

- LONGEST_FLOW_PKT with MAX_IP_PKT_LEN;
- MIN_TTL with MAX_TTL;
- TCP_FLAGS with CLIENT_TCP_FLAGS;
- RETRANSMITTED_OUT_BYTES with RETRANSMITTED_OUT_PKTS;
- OUT_BYTES with NUM_PKTS_1024_TO_1514_BYTES.

For each feature in the pair, the cumulative value of all absolute correlation coefficient values is calculated. The higher the cumulative correlation value, the less helpful the feature. This way, one receives sufficient information on removing unneeded features—one from each pair. In line with that, the following features are rejected: LONGEST_FLOW_PKT, MAX_TTL, CLIENT_TCP_FLAGS, RETRANSMITTED_OUT_PKTS, and NUM_PKTS_1024_TO_1514_BYTES. To summarize, 32 features, containing 24 numerical and 8 categorical features, are left for further analysis after this stage.

4.2. Classification and Feature Selection Algorithms

Three tree-based classifiers (random forests, extra trees, and adaboost), as well as three linear models (passive aggressive algorithm, support vector classifier, and ridge classifier) are utilized to find the best features. All algorithms used in this research are examples of embedded feature selection methods.

Random forest (RF) is a batch of decision trees [49] trained on the randomly selected subspaces of training data [50]. At each tree node, the following steps are repeated: choose random features from the set of all features and split, among the best ones, into the daughter nodes [51]. The best feature is selected according to the outcome of the criterion function. This action is finished when the boundary conditions are met, e.g., reaching the number of different class samples in a node. Similarly, in the testing process, each tree is tested on a random part of the testing data. The outcome of the classifier is nothing but aggregating all outcomes of particular trees. Extra trees (ET) have much in common with random forests except for two conditions. Firstly, each tree is trained on all training data. Then, in each tree node, the splitting is performed randomly. The remaining parts of the algorithm are ideally the same as in the previously mentioned random forests [52]. The Adaboost (AB) classifier brings together a batch of weak decision trees, where each one is trained on a subspace of training data. Each data sample misclassified by a tree obtains a higher weight than other samples [53]. Then, each subsequent decision tree is built on a base of previously updated weights. On top of that, each tree receives its score, which indicates its importance in the final result of the classification process [54]. In tree-based classifiers, the importance of the particular feature is a summed importance measure from all trees' nodes where this feature occurs [51].

The passive aggressive algorithm (PA) is very similar to the concept of the perceptron algorithm, where the model's weight vector is updated with an analysis of every new training sample [55]. The noticeable difference during training is that in the passive aggressive model, the weight vector is not only updated when a new sample is badly classified. The algorithm's aggressiveness also updates the weight vector when the result is correct, but the value of the loss function is not equal to zero. On the other hand, when the value of the loss function is equal to zero, the weights are not altered. This mechanism is the so-called passiveness [56]. Additionally, for noisy data, a special parameter is introduced that controls the trade-off between these two terms [57]. The support vector classifier (SVM) converts the space of features and finds linear boundaries to differentiate different classes of samples [51]. To improve the performance of the classifier, a popular optimization technique of stochastic gradient descent [58] is used. The ridge classifier (R) is a special case of ridge regression. This method converts input data into a feature space, in which the linear regression function is utilized [59]. This method is based on the SVM concept [60]. The feature importance measure in linear classifiers is obtained much differently from the tree-based ones. In this case, one should analyze the feature weights calculated during the algorithms' training phase. The higher the absolute value of a feature's weight in the model, the more important the feature [61].

4.3. Features Ranking

The concept of this examination is to find the best set of features on the basis of the joint results of six different algorithms. In other words, the outcome of many feature rankings is combined to construct one final ranking.

A batch of systematic experiments was performed to create feature rankings for each classifier. The aim of these experiments was to find optimal hyperparameters of each classifier, maximizing the chosen performance measure. For this purpose, the grid search approach was used. After each experiment, its' evaluation metrics were saved. Then, all approaches for a chosen classifier were compared in order to pick the one with the highest classification results. This phase of the research is displayed on the left side of Figure 1. The very first set of boxes on this chart represents all random forest tests, among which the most efficient one was chosen to focus on creating its ranking of features—"Ranking of the best RF classifier". A similar procedure was used to find rankings of features of the best variant of all other classifiers.

As a final result of tests with six classifiers, described in the previous section, one has six optimized (i.e., with a set of parameters that offers the highest classification quality) classifiers. The validation of each classifier is performed with one of three evaluation measures by computing the test dataset. The three measures, accuracy, recall, and precision, are explained in Equation (1):

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad recall = \frac{TP}{TP + FN}, \quad precision = \frac{TP}{TP + FP} \quad (1)$$

where TP stands for the number of all samples from a specific class correctly classified, TN is the number of all samples from other classes correctly classified as not belonging to the specific class, FP is the number of all samples from other classes wrongly classified as the specific class, and FN stands for the number of samples of a specific class that are not classified as this class but as any other class.

Apart from the final measures describing the quality, these six classifiers are also feature selectors—each of them produces an individual ranking of features. There are the six best rankings: one formed by random forest, one by Adaboost, etc. A weighted average position for each feature was created to build the final ranking. This position was based on the feature's exact positions in all six rankings. The weight of each component ranking position in these calculations is the accuracy of the classifier producing the given component ranking (see Figure 1):

$$r_i = \sum_{j=1}^6 (r_{max} - r_{i,j}) \cdot q_j, \quad (2)$$

where r_i stands for the final inverted ranking of the i -th feature, $r_{i,j}$ is the component ranking of the i -th feature produced by the j -th classifier, r_{max} is the maximum rank value, and q_j is the quality measure (accuracy, precision, or recall) j -th among 6 classifiers ($j = 1, \dots, 6$).

Due to the inversion of ranks in Equation (2), the final weighted ranking is also inverted. So, at the end, features are sorted in descending order. The sorted list is an ultimate ranking of features, where the lower an index of features, the higher its importance.

The novelty of this study, compared to the reviewed literature, is the use of a set of optimal classifiers to rank features, which, in this case, consists of six models. The selection of optimal classifiers involves checking which tuned parameters achieve the highest evaluation metrics.

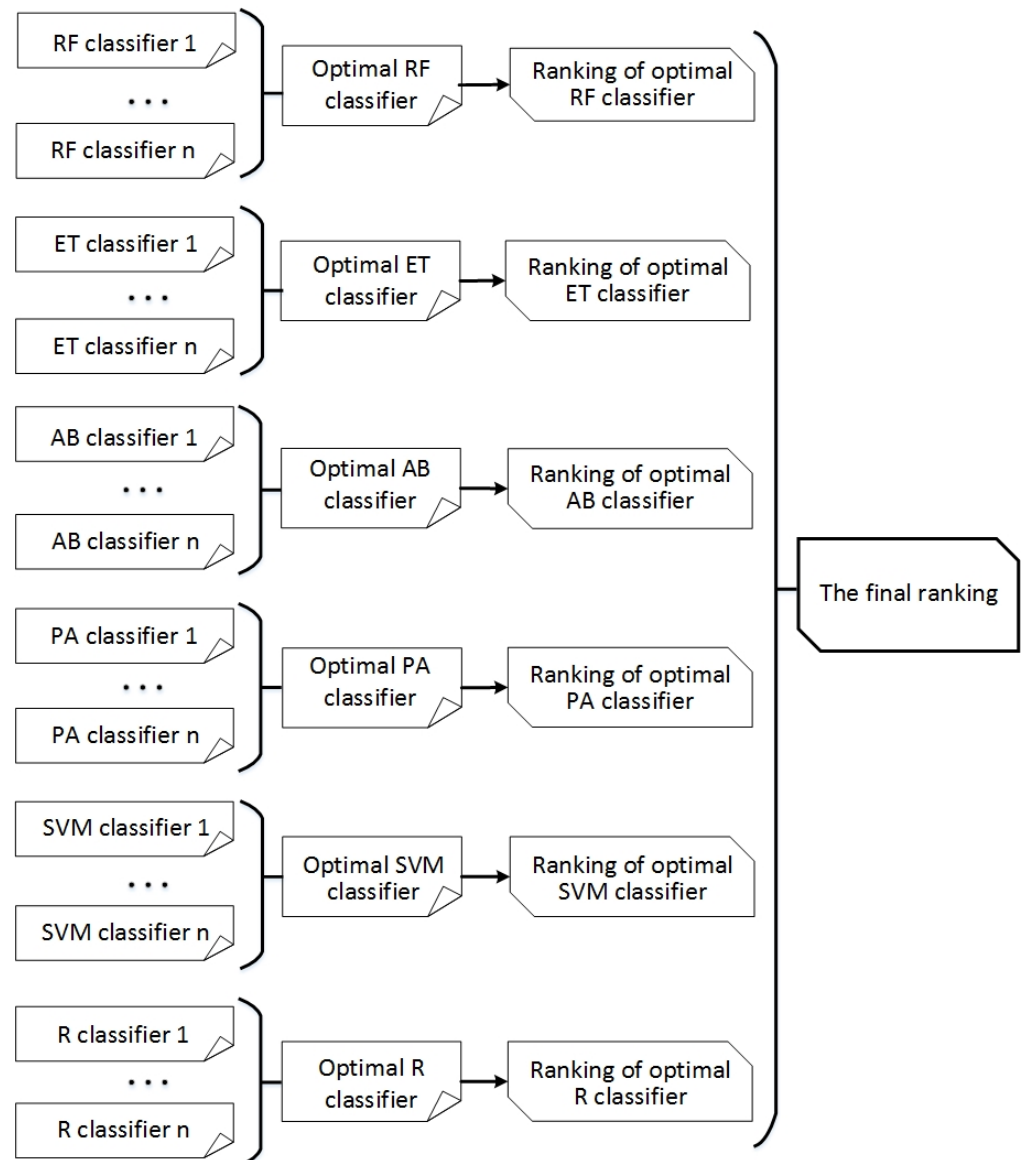


Figure 1. Ranking scheme.

5. Tests

This section contains the research results. At the beginning, the method of selecting the parameters of the classifiers is presented. Then, the obtained rankings of features are shown. The last part reports the related topic of the time needed for classification.

5.1. Choice of Optimal Classifiers

While tuning up each of the six classifiers, a batch of tests was conducted. The grid search approach was used to find an optimal combination of the classifier hyperparameters. The results of this examination can be seen in Table 4. For each classifier, all combinations of the parameters from the column “The checked values” were consequently checked. According to the table, a random forest with 250 trees and 10 as the maximum depth of each tree achieved the highest classification results. When it comes to the Adaboost, the best evaluation metrics were for 200 trees with SAMME, which is a popular boosting algorithm [54].

Table 4. The summary of the choice of the optimal parameters for six classifiers in the highest-accuracy scenario. Abbreviations that stand for classifiers are introduced in Section 4.2.

	Tuned Parameter	Optimal Value	Tested Values
RF	The number of trees	250	{50, 100, 150, 200, 250}
RF	Max. depth of the tree	10	{5, 10}
ET	The number of trees	200	{50, 100, 150, 200, 250}
ET	Max. depth of the tree	10	{5, 10}
AB	The number of trees	200	{50, 100, 150, 200, 250}
AB	The boosting algorithm	SAMME	{SAMME, SAMME.R}
PA	Max. number of iterations	100	{100, 250, 500, 750, 1000}
PA	The regularization (step size)	0.5	{0.5, 1}
SVM	Max. number of iterations	250	{100, 250, 500, 750, 1000}
SVM	The regularization	0.0001	{0.00005, 0.0001}
R	Max. number of iterations	100	{100, 250, 500, 750, 1000}
R	The regularization	1.0	{0.5, 1}

5.2. Feature Ranking Preparation

Having complete rankings, we were able to prepare the final combined ranking of traffic features. High fluctuations of feature ranks in different classifiers were observed. These trends are presented in Figure 2. The placed rankings were built concerning the highest accuracy or recall, as these metrics are equal in this research. The feature ranking of adaboost achieved the lowest accuracy score—80.1%. On the contrary, the highest score was reached by random forest. In the final ranking, due to accuracy, L7_PROTO is the most crucial feature. Interestingly, only one of the classifiers placed that feature as the most important one. The second and third are MIN_TTL and SHORTEST_FLOW_PKT. It is essential to highlight that the final ranking prepared regarding the highest precision is nearly identical to the accuracy ranking. The only differences are the five swaps between the 14th and 18th places.

In Section 3.2, a grouping of features according to the functions they fulfill in network traffic is proposed. The infrastructure independence in which these features were generated is also an important topic of this research. The analysis of Figure 2 shows that the first place in the final ranking is occupied by the feature from group I (L7_PROTO). The following key places are taken by features from groups V (MIN_TTL) and IV (SHORTEST_FLOW_PKT). On the contrary, retransmission features, placed in group VI, occupy noticeably further positions. This outcome confirms that the obtained results are helpful for analyzing network traffic.

5.3. Search for the Minimum Set of Features

The next part of the research is about the issue of reducing the input set of features and preserving high evaluation metrics. As a starting point, the final ranking of features was chosen. The ranking was obtained, just as described in the previous section. As the target classifier, a decision tree was selected. The reason for choosing it was motivated by its simplicity in implementation in real-time systems and its inference speed [7]. This classifier was next tested using the various number of features, starting from all available ones, by reducing, in each step, their number by one. The features were rejected using the inverse order provided by the final ranking. In each iteration, one least significant feature was subtracted. In this examination, the features were ranked according to the final ranking. Then, these results were compared with the classifier that achieved the highest metrics, i.e., random forest.

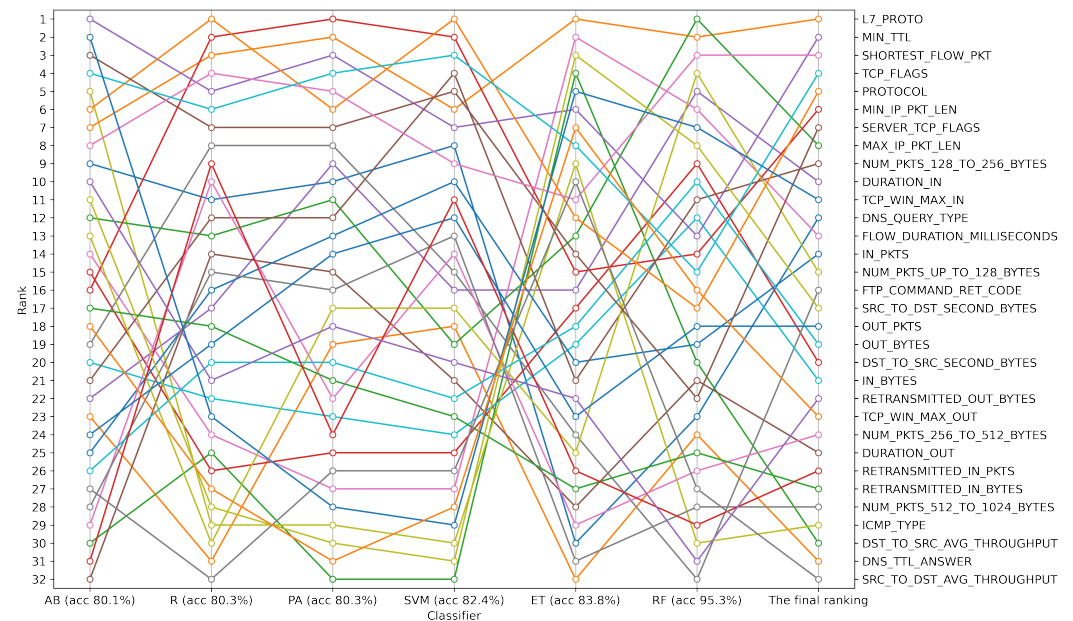


Figure 2. The best rankings due to the accuracy/recall scores. The legend on the right shows the feature order of the final ranking.

In Figure 3, a comparison of accuracy values between the two scenarios is shown. The first one, shown as a blue line, represents the results of the decision tree classification. The orange graph shows the accuracy measure for the best random forest that classifies the best features according to its own feature ranking. In the vast majority of cases, the simple decision tree outperforms the complex random forest. Regarding the nine best features, the decision tree achieves nearly 95.5% accuracy, which is still higher than all random forest metrics. The next intriguing point is the eight best features, where the decision tree achieves 95.3% accuracy against only 94.2% for the random forest. It is the last smallest number of the best features, in which the decision tree outperforms the random forest. The details of the obtained evaluation metrics for the chosen scenarios are displayed in Table 5. The most time-consuming calculations were carried out on Windows 2019, running on a virtual machine with 64 vCores and 484 Gigabytes of RAM. The research used Python's libraries Scikit-learn, NumPy and pandas. In the tests, 90% of the dataset samples were for training and 10% for testing purposes.

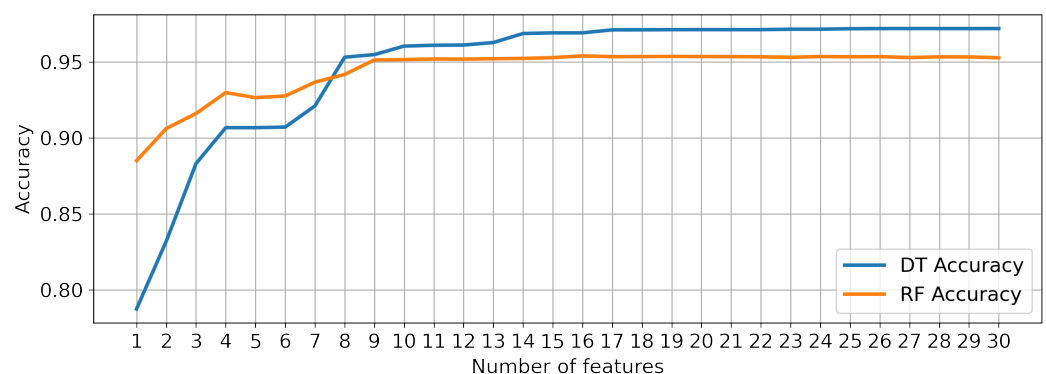


Figure 3. Accuracy and recall scores while removing the least important features for two classifiers: decision tree and random forest.

Table 5. Evaluation metrics for two classifiers: the best random forest and the decision tree.

Features	Accuracy/Recall		Precision	
	Random Forest	Decision Tree	Random Forest	Decision Tree
1	88.5%	78.8%	88.9%	75.6%
2	90.6%	83.2%	91.5%	83.3%
3	91.6%	88.3%	91.0%	88.7%
4	93.0%	90.7%	92.5%	89.9%
5	92.7%	90.7%	92.6%	89.9%
6	92.8%	90.7%	92.7%	90.0%
7	93.7%	92.1%	93.2%	92.4%
8	94.2%	95.3%	93.8%	95.1%
9	95.1%	95.5%	94.7%	95.2%
10	95.2%	96.0%	94.7%	96.0%
15	95.3%	96.9%	94.9%	96.9%
20	95.4%	97.1%	95.1%	97.1%
25	95.3%	97.2%	95.1%	97.2%
30	95.3%	97.2%	95.1%	97.2%

In this work, due to the radically different sizes of the traffic classes, the weighted type of recall and precision metrics are used. This means counting metrics for each class and then calculating a weighted average, where the weight is the percentage of occurrence of a particular class in the whole dataset.

Classification results from the works described in the literature review, as well as the results from this examination, are included in Table 6. The recall and precision metrics were counted as weighted metrics in this research work. In contrast, in most of the papers referenced, there was no precise information on how the metrics for multi-class classification were counted. In addition, the most recent classification results obtained on each of the four subsets of the NF-UQ-NIDS-v2 dataset are added. For each table record, the exact number of processed features is provided.

Table 6. Results obtained in studies of NF-UQ-NIDS-v2 and its subsets.

Method	Dataset	Feat.	Acc.	Rec.	Prec.
RF with Explainable AI [62]	UNSW-NB15	7	96.7%	94.7%	96.1%
Three-level algorithms [63]	part of BoT-IoT	44	100%	100%	100%
Inception Time [64]	ToN-IoT	42	99.7%	99.6%	99.7%
	UNSW-NB15	43	98.6%	98.4%	98.9%
MLP with backpropagation [65]	CIC-CSE-IDS2018	24	99%	98.8%	100%
RF [45]	part of NF-UQ-NIDS-v2	6	99.1%	99%	99%
Autoencoder with MLP [46]	part of NF-UQ-NIDS-v2	42	100%	94.2%	98.9%
Transformer with Neighborhood Clean-ing Rule [47]	NF-BoT-IoT-v2	40	only F1 is provided: 85.6%		
	NF-CSE-CIC-IDS2018-v2	40	only F1 is provided: 88.7%		
Graph Neural Network with Self-supervised learning [48]	part of NF-UQ-NIDS-v2	43	only F1 is provided: 85%		
	part of ToN-IoT	40	only F1 is provided: 98%		
RF [21]	UNSW-NB15-v2	10	100%	100%	100%
	BoT-IoT-v2	10	100%	100%	100%
	ToN-IoT-v2	10	100%	100%	100%
	NF-CSE-CIC-IDS2018-v2	10	100%	100%	100%
	NF-UQ-NIDS-v2	10	98%	98%	98%
Our approach	NF-UQ-NIDS-v2	8	95.3%	95.3%	95.1%

The next phase of the tests is devoted to checking the decision trees' speed. For this purpose, we performed the measurement of their inference times computed on the entire test set—see Table 7. Each value represents the time needed by the corresponding classifier to load the test data and perform the classification. The examination involves classifying test data that represent 10% of the dataset, i.e., nearly 7.6 million samples. It could be seen

that samples with 30 features were tested in 1.86 s, so it is four times longer than just a single-feature sample. Both training samples and testing ones contained the same number of features. For instance, the tree that was tested on ten features was trained beforehand on ten features, etc. The calculations were conducted using Python. The following parameters were used to create decision trees (Scikit-learn): criterion, *log_loss*; splitting strategy, *best*; and minimum number of samples to split a node, 2. As in previous tests, 90% of the given samples were used in the training phase, whereas 10% was used during testing. Figure 4 displays the times needed to classify the given number of features. Each blue dot refers to the classifier built using number of features being its x-coordinate.

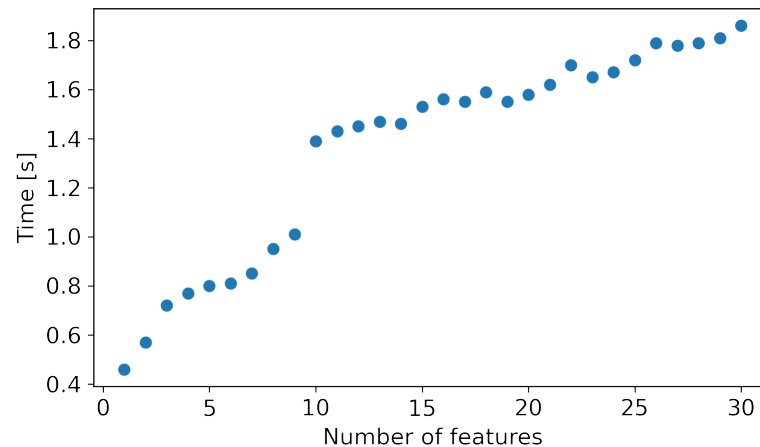


Figure 4. The comparison of DT times needed to test a given number of features.

Analyzing the test time needed for offline classification (see Table 7 and Figure 4), one can see that even for 30 features, it is reasonable. Selecting the optimal features decreases the number of times around 1 s with fewer features. The experiments were conducted on a regular computer with an Intel Core i5 12-gen computer with 16 GB of DDR4 RAM and processor-integrated Intel Iris Xe Graphics. Implementing those algorithms in dedicated hardware instead of software solutions can lead to almost real-time detection systems. An FPGA chip appears to be an interesting solution here. It is worth mentioning that there are works where such a concept has worked successfully. For instance, in paper [66], a decision tree classifier was used on this chip as an IDS. Other compatible solutions are CPU-based (ARM, Intel, Santa Clara, CA, USA) and GPU-based (NVIDIA, Santa Clara, CA, USA) approaches [67,68].

Table 7. Times needed by decision tree for testing.

No. of Feat.	Testing Time [s]	No. of Feat.	Testing Time [s]	No. of Feat.	Testing Time [s]
30	1.86	20	1.58	10	1.39
29	1.81	19	1.55	9	1.01
28	1.79	18	1.59	8	0.95
27	1.78	17	1.55	7	0.85
26	1.79	16	1.56	6	0.81
25	1.72	15	1.53	5	0.80
24	1.67	14	1.46	4	0.77
23	1.65	13	1.47	3	0.72
22	1.70	12	1.45	2	0.57
21	1.62	11	1.43	1	0.46

The times measured for the IDS systems in several research works are shown in Table 8. It is worth noting that some researchers provide the total times needed to train and test their models. There is also work [69], from which one can learn about the times required for analysis only on selected computer attacks. This study is based only on subsets of popular datasets. The analysis of features in different datasets is rather not comparable. For example, the KDD Cup 1999 dataset has binary features (such as information on whether someone has logged in or not), the processing of which is faster than the numerical features.

Table 8. Time listing of selected operations for IDS systems.

Method	Dataset	Feat.	Description	Time [s]
Reduced Error Pruning Tree [69]	part of KDD Cup 1999 part of KDD Cup 1999	19	testing time (only Probe attack)	0.5
		9	testing time (only DoS attack)	0.69
MLP and DT [70]	UGR'16	8	training and testing time	4.25
AB [71]	part of UNSW-NB15	16	training and testing time for only DNS samples	150.8
	part of UNSW-NB15	12	training and testing time for only HTTP samples	148.3
	part of NIMS	16	training and testing time for only DNS samples	142.2
	part of NIMS	12	training and testing time for only HTTP samples	145.6
Autoencoder with MLP [46]	part of NF-UQ-NIDS-v2	42	testing time (train/test split: 80/20)	0.74
	part of NF-UQ-NIDS-v2	42	testing time (train/test split: 70/30)	1.32
	part of NF-UQ-NIDS-v2	42	testing time (train/test split: 60/40)	1.44
Our approach	NF-UQ-NIDS-v2	8	testing time	0.95

6. Conclusions

The research described in this paper focused on finding the minimal set of internet traffic features that preserve the high detection accuracy of harmful internet traffic. Such a feature set allows obtaining a fast decision-tree-based classifier for real-time network intrusion detection systems.

A novel procedure was proposed and applied to rank available features from the NF-UQ-NIDS-v2 dataset to obtain such a set according to their importance for traffic sample classification. In the study described in the paper, the feature selection procedure is based on six classifiers, three that are decision tree based, and three that are linear and optimized, used twofold—first as an actual classifier and second as a tool for feature ordering—resulting in their ranking. Rankings generated by all classifiers are combined in a weighted manner using classifiers' accuracy as ranks to obtain the final ranking.

The second part of this research was focused on investigating the influence of the number of features used to build the classifier on its performance. This is considered both computational efficiency (inference speed) and efficiency in terms of accuracy, precision and recall. As a baseline classifier, the decision tree was chosen. This choice was motivated by its high inference speed—the decision tree is one of the fastest classifiers, being an obvious choice in real-time machine learning-based intrusion detection systems. For comparative purposes, the same analysis was performed on the slower random forest classifier. The random forest outperformed the decision tree when using up to seven features, while when exceeding this number, the simple decision tree performed better, exceeding the level of 95% accuracy. Moreover, the decision-tree classifier built using 8 features, chosen using the proposed procedure, performed $2.5\times$ faster than the one induced using 30 features with only 1.9 p.p. lower accuracy. The obtained results are not higher than those performed on the same dataset and reported in other studies (see Table 6), but it is worth noting that in this study, just a simple classifier, a decision tree, was intentionally selected. The undoubted benefit of this choice is the tree's speed of sample processing, which is faster than that of comparable studies (see Table 8).

The goal of this paper was, however, not only to find an optimal set of features for a particular dataset but also to outline a general approach to processing network traffic data. The proposed framework consists of the creation of a ranking of the features based on ensemble learning followed by weighted feature aggregation. It may be applied to other datasets and based on different classifiers. Moreover, we claim that the results of the feature selection process are more useful when they are expressed not in the form of a single result (a particular number of features for a given classifier) but rather in the form of relations between the number of features and classifier performance in the two above-mentioned aspects. The dependence of the quality metrics and inference speed from the number of features allows one to build a classifier, fulfilling particular requirements. For instance, given a required sample processing speed for a particular network device and transmission standard, one may find the optimal number of features that are used in a decision-tree classifier to guarantee obtaining this speed.

The method proposed in this article is the universal feature ranking approach that may be adapted to particular network setups, trained on various datasets, and employing any chosen classifiers, which is also a possible direction for future research.

Author Contributions: Conceptualization, M.I.; Methodology, M.I. and W.G.; Software, J.K.; Validation, J.K., M.I. and W.G.; Investigation, J.K. and M.I.; Resources, W.G.; Writing—original draft, J.K. and M.I.; Writing—review & editing, J.K., M.I. and W.G.; Supervision, M.I.; Funding acquisition, J.K. and W.G. All authors have read and agreed to the published version of the manuscript.

Funding: The research was funded by POB Cybersecurity and Data Analysis of Warsaw University of Technology within the Excellence Initiative: Research University (IDUB) program.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data supporting the reported results is publicly available at: https://staff.itee.uq.edu.au/marius/NIDS_datasets/ (accessed on 5 August 2024).

Acknowledgments: We are grateful for the support from CloudFerro company. A significant part of this research calculations was conducted on the company's resources.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wright, D.; Kumar, R. Assessing the socio-economic impacts of cybercrime. *Soc. Impacts* **2023**, *1*, 100013. [\[CrossRef\]](#)
2. Altulaihan, E.; Almaiah, M.A.; Aljughaiman, A. Anomaly Detection IDS for Detecting DoS Attacks in IoT Networks Based on Machine Learning Algorithms. *Sensors* **2024**, *24*, 713. [\[CrossRef\]](#)
3. Kshirsagar, D.; Kumar, S. Towards an intrusion detection system for detecting web attacks based on an ensemble of filter feature selection techniques. *Cyber-Phys. Syst.* **2023**, *9*, 244–259. [\[CrossRef\]](#)
4. Ashoor, A.S.; Gore, S. Importance of intrusion detection system (IDS). *Int. J. Sci. Eng. Res.* **2011**, *2*, 1–4.
5. Dhal, P.; Azad, C. A comprehensive survey on feature selection in the various fields of machine learning. *Appl. Intell.* **2022**, *52*, 4543–4581. [\[CrossRef\]](#)
6. Thakkar, A.; Lohiya, R. A survey on intrusion detection system: Feature selection, model, performance measures, application perspective, challenges, and future research directions. *Artif. Intell. Rev.* **2022**, *55*, 453–563. [\[CrossRef\]](#)
7. Bouke, M.A.; Abdullah, A.; ALshatebi, S.H.; Abdullah, M.T. E2IDS: An enhanced intelligent intrusion detection system based on decision tree algorithm. *J. Appl. Artif. Intell.* **2022**, *3*, 1–16. [\[CrossRef\]](#)
8. Ingre, B.; Yadav, A.; Soni, A.K. Decision tree based intrusion detection system for NSL-KDD dataset. In *Proceedings of the Information and Communication Technology for Intelligent Systems (ICTIS 2017)-Volume 22, Ahmedabad, India, 25–26 March 2017*; Springer: Berlin/Heidelberg, Germany, 2018; pp. 207–218.
9. Rai, K.; Devi, M.S.; Guleria, A. Decision tree based algorithm for intrusion detection. *Int. J. Adv. Netw. Appl.* **2016**, *7*, 2828.
10. Awad, M.; Fraihat, S. Recursive feature elimination with cross-validation with decision tree: Feature selection method for machine learning-based intrusion detection systems. *J. Sens. Actuator Netw.* **2023**, *12*, 67. [\[CrossRef\]](#)
11. Gudivada, V.; Apon, A.; Ding, J. Data quality considerations for big data and machine learning: Going beyond data cleaning and transformations. *Int. J. Adv. Softw.* **2017**, *10*, 1–20.
12. Guezzaz, A.; Benkirane, S.; Azrour, M.; Khurram, S. A reliable network intrusion detection approach using decision tree with enhanced data quality. *Secur. Commun. Netw.* **2021**, *2021*, 1230593. [\[CrossRef\]](#)
13. Jain, A.; Patel, H.; Nagalapatti, L.; Gupta, N.; Mehta, S.; Guttula, S.; Mujumdar, S.; Afzal, S.; Sharma Mittal, R.; Munigala, V. Overview and importance of data quality for machine learning tasks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Virtual Event, 6–10 July 2020*; pp. 3561–3562.
14. Gupta, N.; Mujumdar, S.; Patel, H.; Masuda, S.; Panwar, N.; Bandyopadhyay, S.; Mehta, S.; Guttula, S.; Afzal, S.; Sharma Mittal, R.; et al. Data quality for machine learning tasks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual Event, 14–18 August 2021*; pp. 4040–4041.
15. Sarhan, M.; Layeghy, S.; Portmann, M. Towards a standard feature set for network intrusion detection system datasets. *Mob. Netw. Appl.* **2022**, *27*, 357–370. [\[CrossRef\]](#)
16. Claise, B. Cisco Systems NetFlow Services Export Version 9—RFC 3954. 2004. Available online: <https://www.rfc-editor.org/info/rfc3954> (accessed on 29 July 2024).
17. Aitken, P.; Claise, B.; Trammell, B. Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information—RFC 7011. 2013. Available online: <https://www.rfc-editor.org/info/rfc7011> (accessed on 29 July 2024).
18. Mostert, W.; Malan, K.M.; Engelbrecht, A.P. A feature selection algorithm performance metric for comparative analysis. *Algorithms* **2021**, *14*, 100. [\[CrossRef\]](#)

19. Chandrashekar, G.; Sahin, F. A survey on feature selection methods. *Comput. Electr. Eng.* **2014**, *40*, 16–28. [[CrossRef](#)]
20. Ferreira, A.J.; Figueiredo, M.A. Efficient feature selection filters for high-dimensional data. *Pattern Recognit. Lett.* **2012**, *33*, 1794–1804. [[CrossRef](#)]
21. Komisarek, M.; Pawlicki, M.; Kozik, R.; Hołubowicz, W.; Choraś, M. How to Effectively Collect and Process Network Data for Intrusion Detection? *Entropy* **2021**, *23*, 1532. [[CrossRef](#)]
22. Honest, N. A survey on Feature Selection Techniques. *GIS Sci. J.* **2020**, *7*, 353–358.
23. Smith, J.; Doe, J. Analysis of Basic Features in Network Traffic for Intrusion Detection. *J. Netw. Secur.* **2020**, *15*, 112–130.
24. Lee, A.; Chen, B. Evaluating Payload Content for Advanced Intrusion Detection. In Proceedings of the International Conference on Cybersecurity, Virtual Event, 26–28 July 2021; pp. 345–356.
25. Kumar, R.; Patel, S. Time-Based Feature Analysis for Real-Time Intrusion Detection. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 987–1001.
26. Martinez, C.; Lopez, S. Behavioral Feature Profiling for Network Intrusion Detection. *J. Comput. Netw.* **2023**, *18*, 215–230.
27. Sharma, Y.; Sharma, S.; Arora, A. Feature ranking using statistical techniques for computer networks intrusion detection. In Proceedings of the 2022 7th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 22–24 June 2022; pp. 761–765.
28. Kumar, A.; Kumar, S. Intrusion detection based on machine learning and statistical feature ranking techniques. In Proceedings of the 2023 13th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, 19–20 January 2023; pp. 606–611.
29. Seijo-Pardo, B.; Bolón-Canedo, V.; Porto-Díaz, I.; Alonso-Betanzos, A. Ensemble feature selection for rankings of features. In *Proceedings of the International Work-Conference on Artificial Neural Networks, Palma de Mallorca, Spain, 10–12 June 2015*; Springer: Berlin/Heidelberg, Germany, 2015; pp. 29–42.
30. He, W.; Li, H.; Li, J. Ensemble feature selection for improving intrusion detection classification accuracy. In Proceedings of the 2019 International Conference on Artificial Intelligence and Computer Science, Wuhan, China, 12–13 July 2019; pp. 28–33.
31. Krishnaveni, S.; Sivamohan, S.; Sridhar, S.; Prabakaran, S. Efficient feature selection and classification through ensemble method for network intrusion detection on cloud computing. *Clust. Comput.* **2021**, *24*, 1761–1779. [[CrossRef](#)]
32. Karimi, Z.; Kashani, M.M.R.; Harounabadi, A. Feature ranking in intrusion detection dataset using combination of filtering methods. *Int. J. Comput. Appl.* **2013**, *78*, 21–27. [[CrossRef](#)]
33. Arora, A.; Peddoju, S.K. Minimizing network traffic features for android mobile malware detection. In Proceedings of the 18th International Conference on Distributed Computing and Networking, Hyderabad, India, 5–7 January 2017; pp. 1–10.
34. Jha, S.K.; Arora, A. An enhanced intrusion detection system using combinational feature ranking and machine learning algorithms. In Proceedings of the 2022 2nd International Conference on Intelligent Technologies (CONIT), Hubli, India, 24–26 June 2022; pp. 1–8.
35. Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. A survey of network-based intrusion detection data sets. *Comput. Secur.* **2019**, *86*, 147–167. [[CrossRef](#)]
36. Krupski, J.; Graniszewski, W.; Iwanowski, M. Data Transformation Schemes for CNN-Based Network Traffic Analysis: A Survey. *Electronics* **2021**, *10*, 2042. [[CrossRef](#)]
37. Pinto, A.; Herrera, L.C.; Donoso, Y.; Gutierrez, J.A. Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure. *Sensors* **2023**, *23*, 2415. [[CrossRef](#)] [[PubMed](#)]
38. Pavlov, A.; Voloshina, N. Dataset Selection for Attacker Group Identification Methods. In Proceedings of the 2021 30th Conference of Open Innovations Association FRUCT, Oulu, Finland, 27–29 October 2021; pp. 171–176.
39. Ahmed, L.A.H.; Hamad, Y.A.M.; Abdalla, A.A.M.A. Network-based Intrusion Detection Datasets: A Survey. In Proceedings of the 2022 International Arab Conference on Information Technology (ACIT), Abu Dhabi, United Arab Emirates, 22–24 November 2022.
40. De Keersmaecker, F.; Cao, Y.; Ndonga, G.K.; Sadre, R. A survey of public IoT datasets for network security research. *IEEE Commun. Surv. Tutor.* **2023**, *25*, 1808–1840. [[CrossRef](#)]
41. Moustafa, N.; Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, Australia, 10–12 November 2015; pp. 1–6.
42. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [[CrossRef](#)]
43. Alsaedi, A.; Moustafa, N.; Tari, Z.; Mahmood, A.; Anwar, A. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access* **2020**, *8*, 165130–165150. [[CrossRef](#)]
44. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the International Conference on Information Systems Security and Privacy, Funchal, Portugal, 22–24 January 2018; pp. 108–116.
45. Gouda, H.A.; Ahmed, M.A.; Roushdy, M.I. Optimizing anomaly-based attack detection using classification machine learning. *Neural Comput. Appl.* **2024**, *36*, 3239–3257. [[CrossRef](#)]
46. Adeniyi, O.; Sadiq, A.S.; Pillai, P.; Aljaidi, M.; Kaiwartya, O. Securing Mobile Edge Computing Using Hybrid Deep Learning Method. *Computers* **2024**, *13*, 25. [[CrossRef](#)]

47. Qing, Y.; Liu, X.; Du, Y. Mitigating data imbalance to improve the generalizability in IoT DDoS detection tasks. *J. Supercomput.* **2023**, *80*, 9935–9960. [\[CrossRef\]](#)
48. Gu, Z.; Lopez, D.T.; Alrahis, L.; Sinanoglu, O. Always be Pre-Training: Representation Learning for Network Intrusion Detection with GNNs. In Proceedings of the 2024 25th International Symposium on Quality Electronic Design (ISQED), San Francisco, CA, USA, 3–5 April 2024; pp. 1–8.
49. Ho, T.K. Random decision forests. In Proceedings of the 3rd International Conference on Document Analysis and Recognition, Montreal, QC, Canada, 14–16 August 1995; Volume 1, pp. 278–282.
50. Louppe, G.; Wehenkel, L.; Sutura, A.; Geurts, P. Understanding variable importances in forests of randomized trees. *Adv. Neural Inf. Process. Syst.* **2013**, *26*.
51. Hastie, T.; Tibshirani, R.; Friedman, J.H.; Friedman, J.H. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*; Springer: Berlin/Heidelberg, Germany, 2009; Volume 2.
52. Geurts, P.; Ernst, D.; Wehenkel, L. Extremely randomized trees. *Mach. Learn.* **2006**, *63*, 3–42. [\[CrossRef\]](#)
53. Freund, Y.; Schapire, R.E. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. Syst. Sci.* **1997**, *55*, 119–139. [\[CrossRef\]](#)
54. Hastie, T.; Rosset, S.; Zhu, J.; Zou, H. Multi-class adaboost. *Stat. Its Interface* **2009**, *2*, 349–360. [\[CrossRef\]](#)
55. Freund, Y.; Schapire, R.E. Large margin classification using the perceptron algorithm. In Proceedings of the Eleventh Annual Conference on Computational Learning Theory, Madison, WI, USA, 24–26 July 1998; pp. 209–217.
56. Hoi, S.C.; Sahoo, D.; Lu, J.; Zhao, P. Online learning: A comprehensive survey. *Neurocomputing* **2021**, *459*, 249–289. [\[CrossRef\]](#)
57. Crammer, K.; Dekel, O.; Keshet, J.; Shalev-Shwartz, S.; Singer, Y. Online passive aggressive algorithms. *J. Mach. Learn. Res.* **2006**, *7*, 551–585.
58. Zhang, T. Solving large scale linear prediction problems using stochastic gradient descent algorithms. In Proceedings of the Twenty-First International Conference on Machine Learning, Banff, AB, Canada, 4–8 July 2004; p. 116.
59. Saunders, C.; Gammerman, A.; Vovk, V. Ridge regression learning algorithm in dual variables. In Proceedings of the 15th International Conference on Machine Learning, Madison, WI, USA, 24–27 July 1998.
60. Drucker, H.; Burges, C.J.; Kaufman, L.; Smola, A.; Vapnik, V. Support vector regression machines. *Adv. Neural Inf. Process. Syst.* **1996**, *9*.
61. Molnar, C. *Interpretable Machine Learning*, 2nd ed.; Lulu. com: Morrisville, NC, USA, 2022.
62. Larriva-Novo, X.; Sánchez-Zas, C.; Villagrà, V.A.; Marín-Lopez, A.; Berrocal, J. Leveraging Explainable Artificial Intelligence in Real-Time Cyberattack Identification: Intrusion Detection System Approach. *Appl. Sci.* **2023**, *13*, 8587. [\[CrossRef\]](#)
63. Alosaimi, S.; Almutairi, S.M. An intrusion detection system using BoT-IoT. *Appl. Sci.* **2023**, *13*, 5427. [\[CrossRef\]](#)
64. Tareq, I.; Elbagoury, B.M.; El-Regaily, S.; El-Horbaty, E.S.M. Analysis of ton-iot, unw-nb15, and edge-iiot datasets using dl in cybersecurity for iot. *Appl. Sci.* **2022**, *12*, 9572. [\[CrossRef\]](#)
65. Alzughairi, S.; El Khediri, S. A cloud intrusion detection systems based on dnn using backpropagation and pso on the cse-cic-ids2018 dataset. *Appl. Sci.* **2023**, *13*, 2276. [\[CrossRef\]](#)
66. Sobh, T.S.; Amer, M.I. Fpga-based network traffic security: Design and implementation using c5.0 decision tree classifier. *J. Electron. Sci. Technol.* **2013**, *11*, 393–403.
67. Abdulhammed, R.; Faezipour, M.; Elleithy, K.M. Network intrusion detection using hardware techniques: A review. In Proceedings of the 2016 IEEE Long Island Systems, Applications and Technology Conference (LISAT), Farmingdale, NY, USA, 29 April 2016; pp. 1–7.
68. Ngo, D.M.; Lightbody, D.; Temko, A.; Pham-Quoc, C.; Tran, N.T.; Murphy, C.C.; Popovici, E. HH-NIDS: Heterogeneous hardware-based network intrusion detection framework for IoT security. *Future Internet* **2022**, *15*, 9. [\[CrossRef\]](#)
69. Tchakoucht, T.A.; Ezziyyani, M. Building a fast intrusion detection system for high-speed-networks: Probe and dos attacks detection. *Procedia Comput. Sci.* **2018**, *127*, 521–530. [\[CrossRef\]](#)
70. Larriva-Novo, X.; Vega-Barbas, M.; Villagrà, V.A.; Rivera, D.; Alvarez-Campana, M.; Berrocal, J. Efficient distributed preprocessing model for machine learning-based anomaly detection over large-scale cybersecurity datasets. *Appl. Sci.* **2020**, *10*, 3430. [\[CrossRef\]](#)
71. Moustafa, N.; Turnbull, B.; Choo, K.K.R. An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. *IEEE Internet Things J.* **2018**, *6*, 4815–4830. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.